VisualLogic

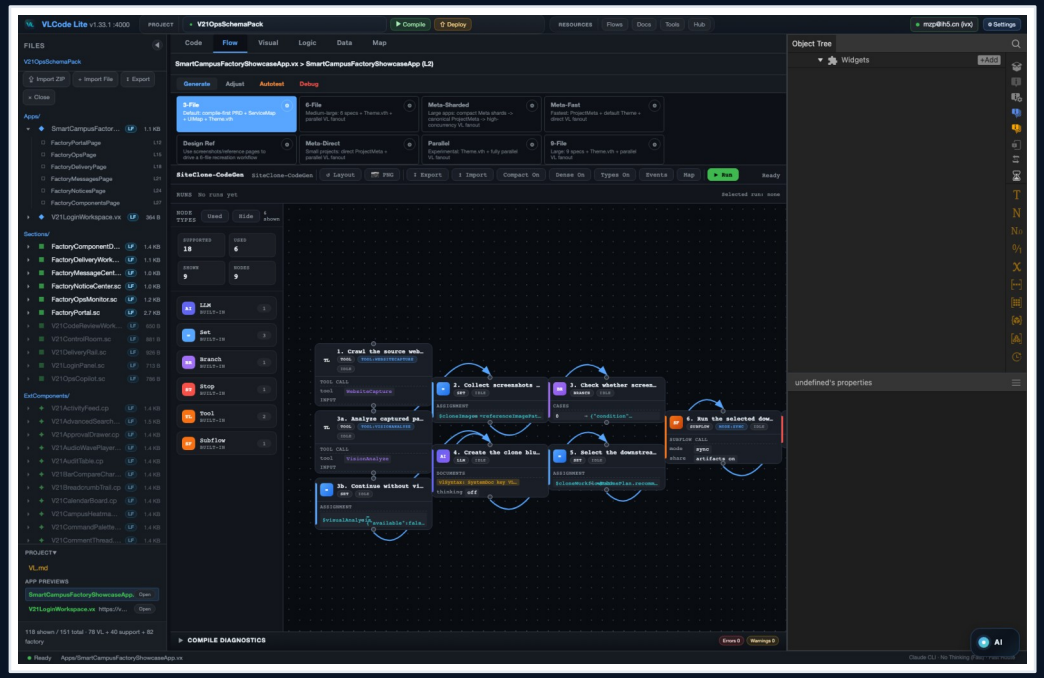
ENTERPRISE AI ASSETS · LOCAL-FIRST

Own the intelligence.
Don't rent it.

An AI asset must outlive any model and stay inside the enterprise. The carrier that makes it concrete: a .vflow — the whole workflow that solves one problem — maintained and run as a local-first Agent App.

Open source · Local-first — assets stay inside the enterprise

github.com/VisualLogic-AI/VL-Code



An AI asset, made concrete — a .vflow that runs as a DAG.

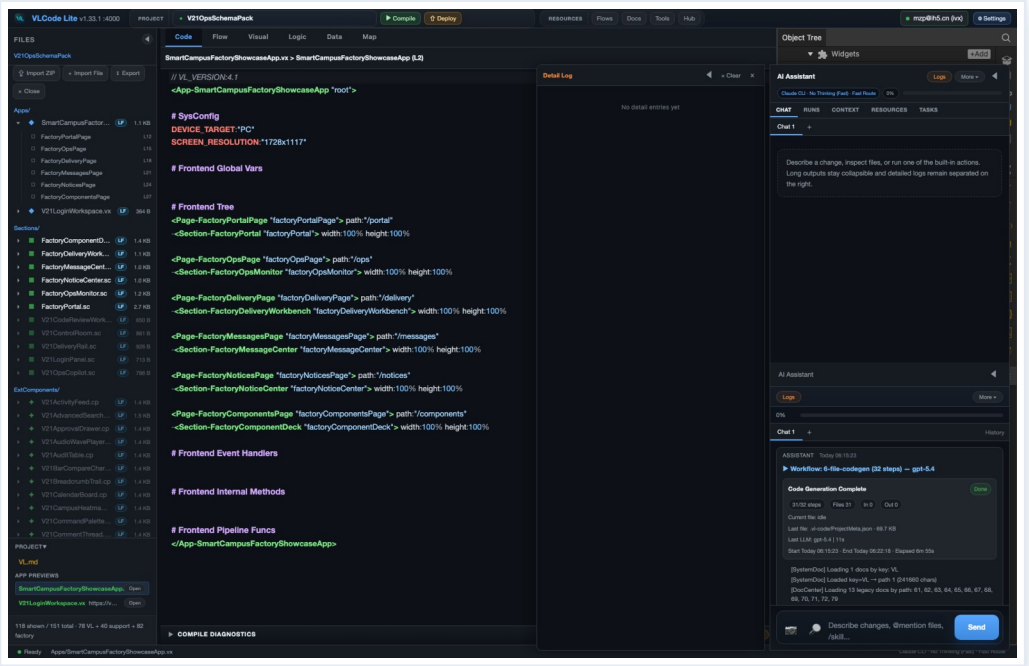
01 THE GAP

A game needs a prompt. An app needs a harness.

AI coding can make a game from one sentence — so why not durable enterprise value?

A game can live inside the prompt — rules, assets, state, and feedback are self-contained. A real enterprise solution cannot: it must connect backend resources, data contracts, approvals, and human decisions across many steps. It is not that AI cannot write the code; harnessing a real project is a long, grounded process — not one giant super-node. That long, grounded process is where today's AI value evaporates: it lives in throwaway chats and one person's prompts, and nothing accumulates.

> ***One AI node creates output.*** ***A DAG harness creates progress.***



Chat alone: output, but no asset.

02 THE DEFINITION

An AI asset. Not a chat log.

Most AI adoption leaves nothing durable. An enterprise AI asset is the opposite — it meets four conditions.

Decoupled from the model

Pluggable providers, right-model-per-node — the model is a swappable input, not the asset. The value lives in the representation, which outlives any model version.

Accumulates your own

It banks the enterprise's own knowledge, methods, AI capabilities, and production experience — generate once, store it, reuse it, instead of re-buying it each time.

Kept inside the enterprise

Local-first means the asset never has to leave — Lint / Compile / codegen run offline. Cloud deploy is an option, not a cage; export the React+Node source = 0 lock-in.

A solution, carried by Flow

A complete solution — or a sharp one to a specific problem. Knowledge, docs (including skills), code and components, orchestrated into one Workflow / DAG: a tamper-evident .vflow bundle.

03 PILLAR 1 · MODEL-DECOUPLED

The model is an input. Not the asset.

Swap the model — the asset survives intact.

Pluggable providers, right model per node

Anthropic, OpenAI, DeepSeek, Kimi, GLM, Gemini — all swappable. Expensive models for the hard nodes, cheap ones for the routine. The model plugs in; swapping it never destroys the asset.

VL is the IR — value that outlives any model

Representation efficiency > model size. The asset lives in the VL representation — human intent compiled to executable code — not in whichever model generated it. Whoever controls the representation controls the ceiling.

04 PILLAR 2 · PRECIPITATE IN-HOUSE

Keep it in-house. Compound it.

The asset never leaves the building — and grows every time you use it.

Local-first, zero lock-in

VL Agent is local-first, not SaaS. Assets stay on your machines; export the source any time. The enterprise owns the intelligence — not a vendor.

Bank the token-compute

The Component Factory stores expensive token-compute: generate once, reuse forever. Don't buy more GPUs — store the compute, share it to everyone.

Govern who uses what

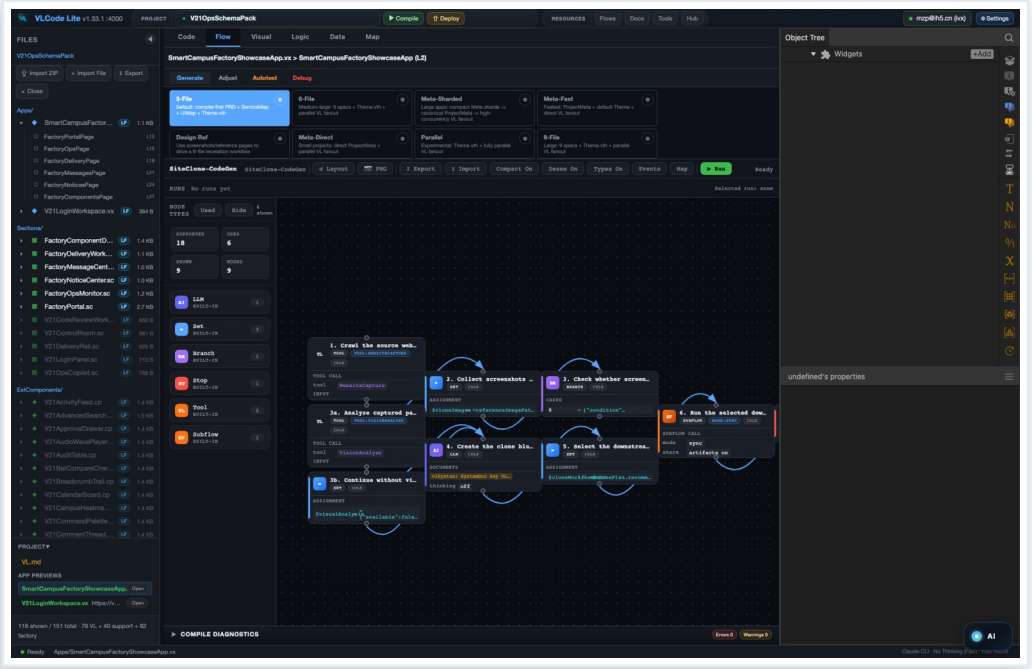
The Resource Center runs a review-gated lifecycle — local → cloud-private → submitted → platform-shared. Nothing reaches the shared inventory without review.

05 PILLAR 3 · FLOW IS THE CARRIER

A solution isn't a prompt. It's a Flow.

Knowledge + docs (incl. skills) + code + components, orchestrated to solve one problem — that is exactly a Workflow / DAG.

A real enterprise solution must connect resources, steps, validation, and human decisions across a long line — and AI is brilliant at a point but can't hold a line. A DAG is the bridge from point intelligence to process intelligence: creativity in the nodes, determinism in the edges. So the natural carrier of a complete or problem-specific solution is a Flow — the one bundle that holds the whole line, not a clever sentence that holds none of it.



Flow mode — a solution as a verifiable DAG.

> ***A game needs a prompt. A solution needs a Flow.***

06 PILLAR 3 · THE ASSET IS A .VFLOW

The asset IS a .vflow

Not a black box. One self-describing bundle that carries the whole solution.

Portable & plain

A ZIP: the complete Workflow JSON (a steps DAG, Spec 4.1), its subflows, an auto-generated resource manifest, prompts and docs — in plain folders, not a black box.

Declares its portability

self-contained / needs-runtime / needs-secrets — you know before you open it whether it will just run.

Tamper-evident

Every file is sha256-checksummed against the manifest; a mismatched or missing file is rejected on import. Checksums, not signatures.

A .vflow is the asset itself — plain, portable, verifiable.

07 PILLAR 4 · MAINTAIN

Maintain it visually.

Everything is a picture you can inspect — and any run you can pause, resume, and audit.

Architecture → Map

The whole asset as one architecture graph — see how every part connects at a glance.

Process → DAG

Each run is a verifiable steps DAG: set breakpoints, pause, resume, rerun from any step.

Data → ER

Data modeled as an ER graph; code ↔ graph is 1:1 bidirectional — edit either side.

Everything graphical · the IDE is a projection of the language.

08 PILLAR 4 · RUN

Same file. Now an Agent App.

Add one app block — the same .vflow becomes installable. Not a separate .vapp file.

1

Install

An entry workflow plus a top-level app block makes one installable, portable Agent App — install, start, pause, resume.

2

Run

The flow's own human-in-the-loop steps become the app's interaction cards automatically. Manifest runs, the Workflow Engine executes, VL renders.

3

Audit

A SQLite control plane records every run, task, artifact, and review — so any run is replayable and auditable.

09 THE ASSET · TWO SHAPES

Two shapes. One compounding library.

An AI asset is a complete solution — or a solution to one specific problem. Both ship as a .vflow.

A complete solution

An end-to-end Agent App as one .vflow — like the in-box Approval Flow Runtime: requester and approver cards, SLA reminders, durable background execution. The whole solution, in one portable file.

A problem-specific solution

One sharp capability as a .vflow — knowledge, docs (incl. skills), code and components orchestrated to solve a single problem. Drop it into your library; import it into VLCode when you need it.

> ***Generate once, reuse forever — the enterprise's intelligence compounds.***

10 GOVERNANCE & AUDIT

Not a file. A governed asset.

Trustworthy by construction — every run replayable, every change reviewed, every byte checked.

Control-plane ledger

A SQLite control plane records every run, task, artifact, and review. The ledger is what turns a clever one-off into an institutional asset.

Review-gated lifecycle

Review before ship. Resources move local → cloud-private → submitted → platform-shared — govern who can use what at every gate.

Tamper-evident bundle

Every file in the .vflow is sha256-checksummed against its manifest; a mismatched or missing file is rejected on import — byte-for-byte across both products.

Integrity is sha256 checksums — tamper-evident, not signed or encrypted.

11 THE WHOLE THESIS

Don't rent intelligence by the token. Own it as an asset.

Four pillars, one carrier: model-decoupled (pluggable providers, right model per node) · precipitated in-house (knowledge, methods, AI capability, production experience) · carried by Flow (a .vflow bundles the whole solution) · maintained and run as a local-first Agent App.

> ***The model is a swappable input. The asset is what outlives it — and stays inside the enterprise.***

THE BEST HOME

Own the intelligence. Don't rent it by the token.

A workflow .vflow — maintained and run as a local-first Agent App — is the best home for enterprise AI assets.
Decoupled from the model · kept inside the enterprise · compounding.

github.com/VisualLogic-AI/VL-Code · **github.com/VisualLogic-AI/VLCode-Lite-agent-os**